

```

#include <hardware/gpio.h>
#include "hardware/adc.h"
#include <pico/stdlib.h>
#include <pico/time.h>
#include <hardware/pwm.h>
#include <time.h>
#include <stdlib.h>
#include <stdio.h>

/* Pin number defines */
#define ANGLE_SELECT_PIN 13
#define DATA_PIN 18
#define TIME_SELECT_PIN 19
#define LATCH_PIN 20
#define CLOCK_PIN 21
#define SERVO_PIN 22
#define SERVO_ANALOG 26
#define TIME_ANALOG 28
#define OFFSET_SELECT_PIN
const int DISPLAY_PINS[] = {17, 16, 15, 14};

/* Servo arm angle restrictions (as a percentage) */
#define USER_MINIMUM_ANGLE 10
#define USER_MAXIMUM_ANGLE 170

/* 74HC595 shift register control values */
bool LSBFIRST = 0;
bool MSBFIRST = 1;

/* Servo control values */
typedef struct {
    uint gpio;
    uint slice;
    uint chan;
    uint speed;
    uint resolution;
    bool on;
    bool invert;
} Servo;

Servo mainServo;
int pos = 0;
int setAngle = (USER_MINIMUM_ANGLE + USER_MAXIMUM_ANGLE) / 2;

/* Time control values */
int timer = 0;
int countdown = 0;
int seconds = 0;
int offset = 0;
uint32_t now = 0;
uint32_t stop = 0;

/* Display control values */
int digitValue;
int digits[4];

/* Loop control values */
bool startToggle = false;
bool timeToggle = false;
bool angleToggle = false;

```

```

bool offsetToggle = true;

/* Values that will draw a circle around the display */
struct circle {
    int digit;
    uint8_t shape;
} sections[12] = {
    {3, 0x04},
    {3, 0x08},
    {2, 0x08},
    {1, 0x08},
    {0, 0x08},
    {0, 0x10},
    {0, 0x20},
    {0, 0x01},
    {1, 0x01},
    {2, 0x01},
    {3, 0x01},
    {3, 0x02}
};

/* Hex values representing characters 0 through 9, then A through F */
uint8_t num[] = {0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f,
                 0x77, 0x7c, 0x39, 0x5e, 0x79, 0x71};

/* Prototypes */
void selectDigit(uint8_t value);
void writeData(int value);
void setDigits(int value);
void writeDisplay(int value, int a[]);
void pwmSetDutyH(uint slice_num, uint chan, int d);
void servoInit(Servo *s, uint gpio, bool invert);
void servoOn(Servo *s);
void servoOff(Servo *s);
void servoPosition(Servo *s, uint p);
uint16_t analogRead(uint p);
long map(long x, long in_min, long in_max, long out_min, long out_max);
uint32_t pwm_get_wrap(uint slice_num);
uint32_t pwm_set_freq_duty(uint slice_num, uint chan, uint32_t f, int d);
void shiftOut(uint8_t dataPin, uint8_t clockPin, uint8_t bitOrder, uint8_t value
);

/* Start */
int main(void) {

    /* Initialize Pins */
    stdio_init_all();
    adc_init();
    gpio_init(ANGLE_SELECT_PIN);
    gpio_set_dir(ANGLE_SELECT_PIN, GPIO_IN);
    gpio_pull_up(ANGLE_SELECT_PIN);
    gpio_init(TIME_SELECT_PIN);
    gpio_set_dir(TIME_SELECT_PIN, GPIO_IN);
    gpio_pull_up(TIME_SELECT_PIN);
    gpio_init(LATCH_PIN);
    gpio_set_dir(LATCH_PIN, GPIO_OUT);
    gpio_init(CLOCK_PIN);
    gpio_set_dir(CLOCK_PIN, GPIO_OUT);
    gpio_init(DATA_PIN);
    gpio_set_dir(DATA_PIN, GPIO_OUT);

```

```

    gpio_init(SERVO_PIN);
    gpio_set_function(SERVO_PIN, GPIO_FUNC_PWM);
    adc_gpio_init(SERVO_ANALOG);
    adc_gpio_init(TIME_ANALOG);
    for (int i = 0; i < 4; i++) {
        gpio_init(DISPLAY_PINS[i]);
        gpio_set_dir(DISPLAY_PINS[i], GPIO_OUT);
    }

    /* Initialize servo */
    servoInit(&mainServo, SERVO_PIN, true);
    servoOn(&mainServo);

    /* Initialize potentiometer positions */
    setAngle = map(analogRead(SERVO_ANALOG), 0, 4095, USER_MAXIMUM_ANGLE, USER_MIN
IMUM_ANGLE);
    seconds = map(analogRead(TIME_ANALOG), 0, 4095, 60, 0);

    /* Entire program loop */
    while(1) {
        /* Main keyclicker loop */
        while (!timeToggle && !angleToggle) {
            for (int i = 0, pos = setAngle; pos < setAngle + 6; pos += 1, i++) {
                servoPosition(&mainServo, pos);
                for (int j = 0; j < 4; j++) {
                    selectDigit(sections[i].digit);
                    writeData(sections[i].shape);
                    now = to_ms_since_boot(get_absolute_time());
                    sleep_ms(5);
                }
            }
            for (int i = 6, pos = setAngle; pos > setAngle - 6; pos -= 1, i++) {
                servoPosition(&mainServo, pos);
                for (int j = 0; j < 4; j++) {
                    selectDigit(sections[i].digit);
                    writeData(sections[i].shape);
                    sleep_ms(5);
                }
            }
            if (offsetToggle == true) {
                srand((unsigned) time(NULL));
                offset = (rand() % 2000);
            }
            else
                offset = 0;
            now = to_ms_since_boot(get_absolute_time());
            timer = (now + (seconds * 1000) + offset);

            while (to_ms_since_boot(get_absolute_time()) < (now + (seconds * 1000) + o
ffset)) {
                countdown = timer - to_ms_since_boot(get_absolut
e_time());

                digits[0] = ((countdown % 1000000) / 100000);
                digits[1] = ((countdown % 100000) / 10000);
                digits[2] = ((countdown % 10000) / 1000);
                digits[3] = ((countdown % 1000) / 100);
                for (int i = 0; i < 4; i++) {
                    selectDigit(i);
                    digitValue = digits[i];
                    if (i == 2) {

```

```

        writeData(num[digitValue] | 0x80);
    }
    else {
        writeData(num[digitValue]);
    }
    sleep_ms(5);
    writeData(0x00);
}
for (int i = 0; i < 4; i++) {
    selectDigit(i);
    writeData(0x00);
}
if (!gpio_get(TIME_SELECT_PIN)) {
    sleep_ms(20);
    if (!gpio_get(TIME_SELECT_PIN)) {
        timeToggle = true;
        startToggle = false;
        break;
    }
}
if (!gpio_get(ANGLE_SELECT_PIN)) {
    sleep_ms(20);
    if (!gpio_get(ANGLE_SELECT_PIN)) {
        angleToggle = true;
        startToggle = false;
        break;
    }
}
}
}
sleep_ms(200);

/* Time delay select loop */
while (timeToggle) {
    seconds = map(analogRead(TIME_ANALOG), 0, 4095, 60, 0);
    digits[1] = ((seconds % 1000) / 100);
    digits[2] = ((seconds % 100) / 10);
    digits[3] = (seconds % 10);
    for (int i = 1; i < 4; i++) {
        selectDigit(i);
        digitValue = digits[i];
        writeData(num[digitValue]);
        sleep_ms(5);
        writeData(0x00);
    }
    if (!gpio_get(TIME_SELECT_PIN)) {
        sleep_ms(20);
        if (!gpio_get(TIME_SELECT_PIN)) {
            timeToggle = false;
        }
    }
}

/* Servo arm angle select loop */
while (angleToggle) {
    setAngle = map(analogRead(SERVO_ANALOG), 0, 4095, USER_MAXIMUM_ANGLE, USER_MINIMUM_ANGLE);
    servoPosition(&mainServo, setAngle);
    digits[1] = ((setAngle % 1000) / 100);
    digits[2] = ((setAngle % 100) / 10);

```

```

    digits[3] = (setAngle % 10);
    for (int i = 1; i < 4; i++) {
        selectDigit(i);
        digitValue = digits[i];
        writeData(num[digitValue]);
        sleep_ms(5);
        writeData(0x00);
    }
    if (!gpio_get(ANGLE_SELECT_PIN)) {
        sleep_ms(20);
        if (!gpio_get(ANGLE_SELECT_PIN)) {
            angleToggle = false;
        }
    }
}
}
}

/* Function Definitions */
/*
void pwmSetDutyH(uint slice_num, uint chan, int d) {
    pwm_set_chan_level(slice_num, chan, pwm_get_wrap(slice_num) * d / 10000);
}
*/

void pwmSetDutyH(uint slice_num, uint chan, int d) { // d is 0-10000
    pwm_set_chan_level(slice_num, chan, pwm_get_wrap(slice_num) * d / 10000);
}

void servoInit(Servo *s, uint gpio, bool invert) {
    gpio_set_function(gpio, GPIO_FUNC_PWM);
    s->gpio = gpio;
    s->slice = pwm_gpio_to_slice_num(gpio);
    s->chan = pwm_gpio_to_channel(gpio);

    pwm_set_enabled(s->slice, false);
    s->on = false;
    s->speed = 0;
    s->resolution = pwm_set_freq_duty(s->slice, s->chan, 50, 0);
    pwmSetDutyH(s->slice, s->chan, 250);

    if (s->chan) {
        pwm_set_output_polarity(s->slice, false, invert);
    }
    else {
        pwm_set_output_polarity(s->slice, invert, false);
    }
    s->invert = invert;
}

void servoOn(Servo *s) {
    pwm_set_enabled(s->slice, true);
    s->on = true;
}

void servoOff(Servo *s) {
    pwm_set_enabled(s->slice, false);
    s->on = false;
}

```

```

void servoPosition(Servo *s, uint p) {
    // Convert angle (0-180) to microseconds (500-2500)
    uint16_t microseconds = 500 + (p * 2000) / 180; // p is 0-180°
    // For 50 Hz PWM: period = 20ms = 20000µs
    // Duty cycle = (pulse_width / period) * wrap value
    // If wrap = 20000 (1 count = 1 microsecond at 50 Hz):
    int duty = (microseconds * 10000) / 20000; // 0-10000
    printf("Angle: %d, Duty: %d, Wrap: %d\n", p, duty, s->resolution);
    pwmSetDutyH(s->slice, s->chan, duty);
}

/*
void servoPosition(Servo *s, uint p) {
    printf("Angle: %d, Duty: %d, Wrap: %d\n", p, duty, s->resolution);
    pwmSetDutyH(s->slice, s->chan, p*10+250);
}
*/

uint16_t analogRead(uint p) {
    uint16_t result = 0;
    if (p == SERVO_ANALOG) {
        adc_select_input(0);
        result = adc_read();
    }
    else if (p == TIME_ANALOG) {
        adc_select_input(2);
        result = adc_read();
    }
    else
        printf("Invalid input pin!");
    return result;
}

long map(long x, long in_min, long in_max, long out_min, long out_max) {
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}

void selectDigit(uint8_t value) {
    for (int i = 0; i < 4; i++) {
        gpio_put(DISPLAY_PINS[i], 1); // Clear all 7-segment digits
    }
    gpio_put(DISPLAY_PINS[value], 0); // Open the selected individual 7-seg
    ment display
}

void shiftOut(uint8_t dataPin, uint8_t clockPin, uint8_t bitOrder, uint8_t value)
) {
    uint8_t i;
    for (i = 0; i < 8; i++) {
        if (bitOrder == LSBFIRST) {
            gpio_put(dataPin, !(value & (1 << i)));
        }
        else {
            gpio_put(dataPin, !(value & (1 << (7 - i))));
        }
        gpio_put(clockPin, 1);
        gpio_put(clockPin, 0);
    }
}

```

```

uint32_t pwm_get_wrap(uint slice_num) {
    if (slice_num < 0 || slice_num >= NUM_PWM_SLICES) {
        return 0;
    }
    return pwm_hw->slice[slice_num].top;
}

uint32_t pwm_set_freq_duty(uint slice_num, uint chan, uint32_t f, int d) {
    uint32_t clock = 125000000;
    uint32_t divider16 = clock / f / 4096 + (clock % (f * 4096) != 0);
    if (divider16 / 16 == 0)
        divider16 = 16;
    uint32_t wrap = clock * 16 / divider16 / f - 1;
    pwm_set_clkdiv_int_frac(slice_num, divider16/16, divider16 & 0xF);
    pwm_set_wrap(slice_num, wrap);
    pwm_set_chan_level(slice_num, chan, wrap * d / 100);
    return wrap;
}

void writeData(int value) {
    gpio_put(LATCH_PIN, 0);
    shiftOut(DATA_PIN, CLOCK_PIN, LSBFIRST, value); // Send serial data to 74HC59
5
    gpio_put(LATCH_PIN, 1); // High level will update data to parallel output (74H
C595)
}

```